

MANUAL DE USUARIO

ADXL335 Captura

Aplicación de escritorio para captura y monitoreo
en tiempo real de acelerómetros ADXL335

VERSIÓN

1.0

FECHA

Abril 2026

PLATAFORMA

Windows 10 / 11 · Python 3.11 · PyInstaller 6

INSTITUCIÓN

Universidad del Quindío



Contenido

1	Introducción y requisitos del sistema
2	Instalación y primera ejecución
3	Interfaz de la aplicación
4	Flujo de una sesión de captura
5	Archivos generados y nomenclatura
6	Referencia de campos: archivos CSV
7	Referencia de campos: JSON de sesión
8	Reportes de precheck y summary
9	Códigos de estado y razón
10	Preguntas frecuentes y diagnóstico

Este manual describe cómo instalar y operar la aplicación **ADXL335 Captura**, una herramienta de escritorio para la adquisición y monitoreo en tiempo real de acelerómetros ADXL335 conectados a una placa ESP32 vía USB. El documento explica cada control de la interfaz, el flujo de una sesión de captura, los archivos que la aplicación genera y el significado de cada campo y código de estado.

1 Introducción y requisitos del sistema

ADXL335 Captura permite adquirir, visualizar y almacenar datos de acelerómetros analógicos triaxiales ADXL335 leídos por una ESP32. La aplicación ejecuta un chequeo previo automático, captura la sesión con duración configurable, y al finalizar genera un conjunto completo de archivos de datos y metadatos listos para análisis posterior.

Hardware requerido

Componente	Descripción
Sensor principal	ADXL335 conectado al ADC de la ESP32 (sensor_id = 1, etiqueta sensor_B).
Sensor secundario	Segundo ADXL335 opcional (sensor_id = 2, etiqueta sensor_A).
Microcontrolador	ESP32 con firmware de captura dual o de nodo único.
Interfaz USB	Adaptador CH340, CP210x o similar, con driver de Windows instalado.
Comunicación	Serial a 115200 baud.

Software

La aplicación se distribuye como ejecutable autónomo (ADXL335_Captura.exe). No requiere instalar Python ni dependencias adicionales. El único requisito es **Windows 10 u 11 de 64 bits**. La versión empaquetada utiliza Python 3.11 con PyInstaller 6.

2 Instalación y primera ejecución

Siga estos pasos para dejar la aplicación lista para su uso:

Paso	Acción
1	Extraer ADXL335_Captura_dist.zip en la carpeta deseada, por ejemplo C:\Captura\.
2	Conectar la ESP32 por USB y esperar a que Windows instale el driver automáticamente.
3	Ejecutar ADXL335_Captura.exe. En el primer arranque se crean las carpetas de datos necesarias.
4	No mover el ejecutable de su carpeta: los datos se guardan siempre al lado del .exe.

NOTA

Al primer arranque se crean automáticamente las carpetas data\raw\sensor_B_live\, data\processed\ y reports\analysis_outputs\ junto al ejecutable. No es necesario crearlas manualmente.

3 Interfaz de la aplicación

La ventana principal se divide en dos zonas: el **panel lateral izquierdo**, que contiene la configuración y los indicadores de estado, y el **área central de monitoreo**, donde se muestran las gráficas en tiempo real.

Panel lateral – sección «Sesión»

Control	Descripción
Equipo (COM)	Puerto serial de la ESP32. «Automático» sondea los puertos disponibles y selecciona el correcto. Si hay varios puertos, elija el COM manualmente.
Duración	Tiempo en segundos de la captura principal (10, 20, 30, 60 o 90 s). El precheck es independiente y no cuenta dentro de este tiempo.

Control	Descripción
Nombre de sesión	Etiqueta libre que se incrusta en el nombre de archivo: «ensayo1», «estatico», «vibracion». Admite letras, números y guion bajo. Por defecto «live».
Iniciar	Lanza el ciclo completo: detección de puerto → precheck → captura → guardado. Se deshabilita mientras corre una sesión.
Detener	Cancela la sesión en cualquier fase. Los datos ya capturados se descartan; no se guarda ningún archivo de datos.
Actualizar	Vuelve a escanear los puertos COM disponibles. Útil si conectó la ESP32 después de abrir la aplicación.
Opciones (avanzado)	Muestra configuración avanzada: prefijo de archivo, carpetas de salida, duración del precheck, ventana de gráfica y baud rate. En uso normal no es necesario cambiarlos.

Panel lateral – indicadores de estado

Indicador	Información que muestra
Estado	Estado global: <i>En espera / Preparando / Chequeando / Grabando / Completado / Error.</i>
Conexión	Puerto COM usado y modo de selección (automático o manual).
Principal	Calidad del sensor sensor_id = 1 (sensor_B): pass, revisar o fallo .
Secundario	Calidad del sensor sensor_id = 2 (sensor_A): pass, revisar o fallo .

Área central – gráficas de monitoreo

Gráfica	Descripción
Comparación principal (lg)	Norma del vector de aceleración en tiempo real para ambos sensores. La línea de referencia a 1.0 g indica el valor esperado en reposo.
Detalle por sensor (X, Y, Z)	Tensión en mV de cada eje del sensor seleccionado con el combo «Detalle». Permite detectar saturación o eje muerto antes de que termine la sesión.
Comparativa de actividad (eje Z)	Superpone el eje Z de ambos sensores para comparar si responden de forma similar ante el mismo estímulo mecánico.
Vista ampliada	Botón en la esquina superior derecha. Abre una segunda ventana con las tres gráficas a mayor resolución, útil en pantallas grandes o proyectores.

Tira de métricas rápidas y actividad reciente

Bajo las gráficas se muestran cuatro métricas en vivo: **Monitoreo** (frecuencia de muestreo instantánea en Hz), **Tiempo** (segundos transcurridos desde el inicio), **Muestras** (total de muestras válidas recibidas sumando ambos sensores) y **Salida** (ruta relativa del CSV *processed* generado al finalizar). El registro «Actividad reciente» lista cronológicamente los eventos de la sesión: detección de puerto, inicio y resultado del precheck, inicio de captura y resultado final.

4 Flujo de una sesión de captura

Al presionar **Iniciar**, la aplicación ejecuta cuatro fases en orden. Cada fase actualiza el panel de estado y, en caso de fallo, cancela las siguientes.

Fase	Nombre	Qué hace
Fase 1	Autodetección de puerto	Si «Equipo» está en «Automático», la app escanea todos los COM y sondea cada uno ~1.6 s buscando el encabezado del stream ADXL335. El puerto que responda se selecciona solo. Si hay un único puerto, se usa directamente.

Fase	Nombre	Qué hace
Fase 2	Chequeo previo (precheck dual)	Captura durante el tiempo configurado (por defecto 10 s) con ambos sensores activos. Evalúa: suficientes muestras, sin saturación ADC, sin ejes muertos, sin pérdidas de paquetes excesivas. Si cualquier sensor falla, la sesión se cancela mostrando el motivo.
Fase 3	Captura principal	Graba datos durante el tiempo seleccionado (10–90 s). Las gráficas se actualizan en tiempo real y el sensor principal se muestra en la gráfica de detalle. Al finalizar pasa automáticamente a la fase 4.
Fase 4	Procesamiento y guardado	Calcula la aceleración en g para cada muestra, evalúa la integridad de la captura completa y guarda los archivos <code>_raw.csv</code> , <code>_processed.csv</code> , <code>_session.json</code> y <code>_summary.txt</code> . Actualiza el panel «Resumen» con el estado final.

**IMPORTANT
E**

Si el precheck falla, **no** se generan los archivos `_raw.csv` ni `_processed.csv`. Solo se guarda el `_precheck.txt` con el diagnóstico. Revise la conexión física y vuelva a intentar.

5 Archivos generados y nomenclatura

Cada sesión exitosa genera hasta cinco archivos que comparten el mismo nombre base para facilitar su asociación. El patrón es:

```
{prefijo}_{nombre_sesion}_{AAAAMDD}_{HHMMSS}_{tipo}.{ext}
```

Campo	Significado
{prefijo}	Identificador del experimento. Por defecto <code>sensor_B_live</code> . Se puede cambiar en «Opciones → Prefijo interno».
{nombre_sesion}	Valor del campo «Nombre» en la GUI. Por defecto <code>live</code> . Cámbielo para distinguir ensayos.
{AAAAMDD}	Fecha de inicio de la captura. Ejemplo: <code>20260412</code> = 12 de abril de 2026.
{HHMMSS}	Hora local de inicio en formato 24 h. Ejemplo: <code>134921</code> = 13:49:21.
{tipo}	Sufijo que indica el contenido: <code>raw</code> , <code>processed</code> , <code>session</code> , <code>summary</code> o <code>precheck</code> .
{ext}	<code>.csv</code> para <code>raw</code> y <code>processed</code> , <code>.json</code> para <code>session</code> , <code>.txt</code> para los reportes de texto.

Ejemplo completo de un conjunto de archivos

```
sensor_B_live_ensayo1_20260412_134921_raw.csv
sensor_B_live_ensayo1_20260412_134921_processed.csv
sensor_B_live_ensayo1_20260412_134921_session.json
sensor_B_live_ensayo1_20260412_134921_summary.txt
sensor_B_live_ensayo1_20260412_134921_precheck.txt
```

Ubicación de cada archivo

Sufijo	Carpeta destino	Contenido
<code>_raw.csv</code>	<code>data\raw\sensor_B_live\</code>	Lecturas crudas del firmware.
<code>_processed.csv</code>	<code>data\processed\</code>	Datos con aceleración en g.
<code>_session.json</code>	<code>data\processed\</code>	Metadatos completos de la sesión.
<code>_summary.txt</code>	<code>reports\analysis_outputs\</code>	Resumen de calidad de la captura.
<code>_precheck.txt</code>	<code>reports\analysis_outputs\</code>	Diagnóstico del chequeo previo.

CONSEJO

Todas las rutas son relativas a la carpeta donde está el ejecutable. Si mueve el .exe, mueva también las carpetas data\ y reports\ para que los archivos anteriores sigan accesibles.

6**Referencia de campos: archivos CSV****Archivo _raw.csv**

Contiene exactamente lo que envió el firmware, sin cálculos adicionales. Es la fuente primaria y no debe modificarse. Tiene una fila por muestra recibida de cualquiera de los dos sensores, intercaladas en orden de llegada. El encabezado es:

```
sensor_id, seq, t_us, wall_s, raw_x, raw_y, raw_z, mv_x, mv_y, mv_z
```

Campo	Tipo	Descripción
sensor_id	entero	Identificador del sensor. 1 = principal (sensor_B). 2 = secundario (sensor_A). Permite filtrar filas por sensor al analizar.
seq	entero	Número de secuencia del firmware. Sube de 1 en 1 por sensor. Saltos > 1 entre filas consecutivas del mismo sensor_id indican pérdida de paquetes.
t_us	entero	Marca de tiempo del microcontrolador en microsegundos. Es el reloj interno de la ESP32, no el de la PC. Se usa para calcular la frecuencia real y detectar jitter.
wall_s	float	Tiempo de pared de la PC en segundos desde la primera muestra de la sesión. Permite alinear los datos con eventos externos (video, otros registros).
raw_x / raw_y / raw_z	entero	Lectura cruda del ADC de cada eje (0–4095 para 12 bits). 2048 representa el punto medio (~0 g). Valores ≤ 5 o ≥ 4090 indican saturación.
mv_x / mv_y / mv_z	float	Tensión del eje en mV, convertida como $mv = raw \times (3300 / 4096)$. El punto de reposo del ADXL335 es ~1650 mV (mitad de 3.3 V).

**IMPORTANT
E**

Saturación ADC: valores de raw_x, raw_y o raw_z ≤ 5 o ≥ 4090 indican que la señal física desborda el rango del convertidor. En ese caso las columnas mv y g no son confiables para esas muestras.

Archivo _processed.csv

Contiene los mismos 10 campos del _raw.csv más cuatro columnas adicionales con la aceleración estimada en unidades g. Es el archivo que se usa normalmente para análisis posteriores.

```
... , mv_z, gx_est, gy_est, gz_est, g_norm_est
```

Campo	Tipo	Descripción
gx_est	float	Aceleración estimada del eje X en g: $gx = (mv_x - bias_x) / sens_x$. El bias se estima promediando las primeras muestras. En reposo horizontal debería dar ~0 g.
gy_est	float	Aceleración estimada del eje Y en g. Misma fórmula que gx_est.
gz_est	float	Aceleración estimada del eje Z en g. Si el sensor está plano, la gravedad (~1 g) aparece aquí antes de restar el bias.
g_norm_est	float	Norma euclidiana: $\sqrt{gx^2 + gy^2 + gz^2}$. En reposo debe ser ~1.0 g (solo gravedad). Valores lejos de 1.0 indican mala calibración, movimiento, saturación o eje desconectado.

Cálculo del bias y sensibilidad

En modo por defecto la aplicación aplica un «bias rápido» sin calibración externa. El procedimiento es el siguiente:

Paso	Descripción
1	Se promedian los primeros N valores de mv de cada eje, donde $N = \max(\text{freq_hz}, 1) \times \text{bias_init_s}$. Con $\text{freq} = 100 \text{ Hz}$ y $\text{bias_init_s} = 2 \text{ s}$ se usan 200 muestras.
2	Ese promedio se toma como punto de referencia (bias_mv) por eje.
3	Se aplica la sensibilidad nominal del ADXL335 a 3.3 V: 300 mV/g .
4	$g_x = (\text{mv_x} - \text{bias_x}) / 300$. Idéntico para los ejes Y y Z.
5	Los valores de bias_mv y sens_mv_per_g quedan registrados en el <code>_summary.txt</code> .

7 Referencia de campos: JSON de sesión

El archivo `_session.json` contiene los metadatos completos de la sesión en formato estructurado. Es útil para procesar resultados programáticamente o reproducir exactamente las mismas condiciones de captura. Puede abrirse con cualquier editor de texto.

Clave JSON	Descripción
"config"	Toda la configuración usada: puerto, baud, duración, prefijo, carpetas, sensibilidad y parámetros del precheck. Replica lo configurado en la GUI.
"port_resolution"	Cómo se seleccionó el puerto: modo (auto/manual/sonda), puerto final, inventario de todos los COM disponibles y resultados de sonda.
"metadata_lines"	Lista de comentarios del firmware (líneas que empiezan con #) recibidos al inicio. Contienen versión del firmware y configuración del hardware.
"header_seen"	<code>true</code> si el firmware envió la cabecera CSV antes de los datos. <code>false</code> se puede indicar que la ESP32 ya transmitía al conectar.
"invalid_lines"	Líneas que no pudieron parsearse como datos válidos. Un valor bajo (< 10) es normal; un valor alto sugiere ruido o firmware incompatible.
"seq_jumps"	Salto de secuencia del sensor principal durante la captura. Cada salto representa paquetes perdidos entre ESP32 y PC.
"stream_duration_s"	Duración real de la captura según el reloj de la PC. Puede diferir ligeramente de la duración configurada.
"freq_hz"	Frecuencia de muestreo total estimada en muestras/s (ambos sensores combinados). Para dual a $\sim 100 \text{ Hz}$ por sensor se esperan $\sim 200 \text{ Hz}$.
"sensor_summaries"	Lista con resumen de calidad por sensor: muestras, frecuencia, saltos, mediana de g , saturación, estado y razón.
"capture_integrity"	Resultado del análisis de integridad post-captura. Usa el mismo criterio que el precheck pero sobre la captura real.
"primary_g_info"	Información del cálculo de aceleración del sensor principal: ruta de cálculo, bias_mv aplicado, sensibilidad y muestras usadas para el bias.
"artifacts"	Rutas relativas de todos los archivos generados. Permite saber qué archivos pertenecen a la sesión sin depender de patrones de nombre.

Lectura desde Python

```
import json
data = json.load(open('..._session.json', encoding='utf-8'))
print(data['config']['duration_s'], data['freq_hz'])
```

8 Reportes de precheck y summary

Los archivos `_precheck.txt` y `_summary.txt` son reportes de texto plano con formato «clave: valor». Permiten diagnosticar problemas sin abrir el JSON. El *precheck* se genera siempre que se ejecuta el chequeo previo, incluso si este falla; el *summary* describe la captura principal y solo se genera si la sesión llegó a esa fase.

Campos comunes a ambos reportes

Campo	Descripción
script	Origen del reporte. <code>gui/adxl_live_gui.py</code> indica que fue generado por la app de escritorio.
requested_port	Puerto pedido por el usuario. (auto) si se dejó en automático.
port	Puerto COM finalmente usado. Ejemplo: COM4.
port_resolution_mode	Método de selección del puerto: <code>preferred_port</code> , <code>single_port_inventory</code> , <code>probe_match</code> o <code>usb_inventory_hint</code> .
baud	Velocidad de comunicación serial en bps (normalmente 115200).
samples	Total de muestras recibidas (suma de ambos sensores).
invalid_lines_ignored	Líneas no parseables. Debería ser bajo.
header_seen	true si el firmware envió la cabecera CSV antes de los datos.

Campos específicos del `_precheck.txt`

Campo	Descripción
phase	Siempre <code>dual_integrity_precheck</code> en este archivo.
precheck_duration_s	Duración en segundos de la ventana de chequeo (por defecto 10 s).
precheck_status	<code>pass</code> = todo correcto. <code>fail</code> = al menos un criterio no se cumplió.
precheck_reason	Código(s) que explican el resultado. Ver Sección 9.
sensor_N_label	Etiqueta del sensor N (1: <code>sensor_B</code> , 2: <code>sensor_A</code>).
sensor_N_samples	Muestras recibidas del sensor N durante el precheck.
sensor_N_freq_hz	Frecuencia de muestreo estimada del sensor N en Hz.
sensor_N_seq_jumps	Salto de secuencia en el sensor N (paquetes perdidos).
sensor_N_seq_backtracks	Veces que el número de secuencia retrocedió. > 0 puede indicar reinicio de la ESP32.
sensor_N_t_backtracks	Veces que el timestamp <code>t_us</code> retrocedió. > 0 indica reinicio del reloj del MCU.
sensor_N_max_saturation_pct	Porcentaje máximo de saturación ADC en cualquier eje (<code>raw</code> ≤ 5 o <code>raw</code> ≥ 4090).
sensor_N_max_zero_axis_pct	Porcentaje de muestras con <code>raw</code> = 0 en algún eje. Valor alto → eje desconectado.
sensor_N_max_flatline_142_pct	Porcentaje de muestras con <code>mv</code> = 142. Puede indicar eje muerto con ese valor fijo.
sensor_N_g_norm_median	Mediana de <code> g </code> durante el precheck. En reposo estático es muy baja (~0.01) porque el bias elimina la gravedad dentro de la ventana.
sensor_N_status / sensor_N_reason	<code>pass</code> o <code>fail</code> individual y código de razón del sensor N.

Campos específicos del `_summary.txt`

Campo	Descripción
sensor_id	Identificador de texto del experimento. Ejemplo: <code>sensor_B</code> .
plot_sensor_numeric_id	<code>sensor_id</code> numérico del sensor principal (1 = <code>sensor_B</code>).

Campo	Descripción
second_sensor_numeric_id	sensor_id numérico del sensor secundario (2 = sensor_A).
duration_s	Duración configurada de la captura en segundos.
session_name	Nombre de sesión usado (campo «Nombre» de la GUI).
file_prefix	Prefijo del nombre de archivo.
accel_mode	nominal_quick_g = sensibilidad nominal 300 mV/g. provisional_sensorB_g = sensibilidad personalizada por eje.
stream_duration_s	Duración real de la captura según el reloj de la PC.
freq_hz	Frecuencia total de muestreo (ambos sensores).
seq_jumps	Salto de secuencia en el sensor principal durante la captura.
capture_integrity_status	pass o fail del análisis post-captura.
capture_integrity_reason	Código(s) de razón del análisis post-captura. Ver Sección 9.
bias_mv	Bias estimado en mV para X, Y, Z del sensor principal, separados por comas.
sens_mv_per_g	Sensibilidad usada en mV/g para X, Y, Z.
sensor_N_logic_status	pass, suspect o fail de la calidad por sensor.
sensor_N_logic_reason	Código de razón de calidad del sensor N. Ver Sección 9.
sensor_N_g_norm_median	Mediana de g durante la captura. En reposo ~ 1.0 g.
sensor_N_max_saturation_pct	Saturación máxima durante la captura. > 20 % = señal fuera de rango.
*_relpath	Rutas relativas a los archivos _raw.csv, _processed.csv, _session.json, _precheck.txt y _summary.txt de la sesión.
next_block_input_relpath	Mismo valor que processed_csv_relpath. Indica el archivo de entrada para el siguiente bloque de análisis.

9

Códigos de estado y razón

Los archivos de texto y el JSON usan códigos estandarizados en los campos status, logic_status, precheck_status y los distintos *_reason. Las tablas siguientes explican cada código, su significado y la acción recomendada.

Códigos de estado

Código	Significado	Acción recomendada
pass	Todos los criterios superados.	Ninguna. La sesión es válida y los datos son confiables.
suspect	Datos fuera del rango ideal pero no descartados.	Repetir la sesión. Usar los datos con precaución.
fail	Al menos un criterio crítico no se cumplió.	No usar estos datos. Ver razón y corregir el problema.
completed	Sesión finalizada normalmente (indicador GUI).	Ninguna.
cancelled	Sesión cancelada por el usuario (indicador GUI).	Ninguna.
error	Error inesperado del sistema.	Revisar la conexión y reiniciar la aplicación.

Códigos de razón

Código de razón	Estado	Significado y acción
dual_integrity_ok	pass	Ambos sensores superaron todos los criterios. Datos listos para usar.
integrity_ok	pass	El sensor individual superó todos los criterios.
basic_live_sanity_ok	pass	Verificación básica de sanidad superada en la captura.
few_samples	fail	Menos muestras de las requeridas. Causas: duración muy corta, ESP32 no transmite, baud incorrecto, cable desconectado.
packet_loss_excessive	fail	Más del 10 % de paquetes perdidos. Causas: cable USB de mala calidad, interferencia, monitor serial externo abierto, CPU saturada.
adc_saturation	fail	Más del 20 % de muestras con $raw \leq 5$ o $raw \geq 4090$. El sensor está fuera de rango; revisar alimentación y conexiones.
g_norm_implausible	fail	Mediana de $ g $ fuera de [0.30, 1.70]. Mal orientado, eje muerto o calibración incorrecta.
g_norm_borderline	suspect	Mediana de $ g $ en zona dudosa [0.30–0.60] o [1.40–1.70]. Orientación inusual o perturbación constante.
low_signal_span	suspect	Variación menor a 0.5 mV en algún eje durante la captura. Señal casi plana; sensor inmóvil o con problema.
low_signal_span_static	fail	En el precheck (reposo), la señal varió menos de 0.5 mV. Puede indicar eje muerto o cortocircuito a tierra.
dead_axis_or_disconnected	fail	Más del 95 % de muestras con $raw = 0$ o $mv = 142$ en algún eje. Revisar soldaduras y cables.
counter_reset_detected	fail	El número de secuencia o el timestamp retrocedió: la ESP32 se reinició. Verificar alimentación y firmware.
missing_header	fail	No se recibió la línea de encabezado del CSV. Firmware antiguo o conexión parcial al inicio.
sensor_missing	fail	No se recibió ninguna muestra de ese sensor_id. Revisar la conexión física al ADC.
precheck_disabled	—	El precheck fue desactivado manualmente (modo especial de captura).
not_run	—	El componente aún no se ha ejecutado (estado inicial de la GUI).

10 Preguntas frecuentes y diagnóstico

Pregunta	Respuesta
La app dice «No se encontró el equipo».	La ESP32 no está conectada o Windows no instaló el driver. Verifique en Administrador de dispositivos que aparece un puerto COM al conectar el USB. Si no aparece, instale manualmente el driver CH340 o CP210x.
El precheck falla con sensor_missing.	La ESP32 está conectada pero no llegan datos del sensor que falta. Verifique las conexiones físicas del ADXL335: alimentación 3.3 V y pines XOUT, YOUT, ZOUT a los pines ADC correctos de la ESP32.
El precheck falla con packet_loss_excessive.	Demasiados paquetes perdidos. Pruebe a cambiar el cable USB, cerrar otras aplicaciones que usen el puerto serial (Arduino IDE, monitores serie) y asegúrese de que solo la app esté leyendo el COM.
Los archivos se guardan en un lugar inesperado.	Los datos se guardan siempre junto al .exe. Si movió el ejecutable, los nuevos datos irán a la nueva ubicación; los anteriores permanecen donde estaban.
g_norm_median es ~ 0.01 en el precheck pero ~ 1.0 en el summary. ¿Es normal?	Sí. En el precheck el sensor está en reposo y el bias se estima en esa misma ventana, cancelando la gravedad. En el summary el bias se estima sobre las primeras muestras de la captura; si el sensor no estuvo quieto al inicio, el bias no cancela la gravedad y el resultado es ~ 1.0 g.

Pregunta	Respuesta
¿Cuántas sesiones puedo guardar?	No hay límite en la aplicación: el tope lo fija el espacio en disco. Cada sesión de 60 s genera aproximadamente entre 1 y 3 MB entre todos los archivos.
¿Puedo abrir los CSV en Excel?	Sí. Son CSV estándar con separador de coma. En Excel: <i>Datos</i> → <i>Obtener datos</i> → <i>Desde texto/CSV</i> , seleccione el archivo y use coma como delimitador.
¿Qué diferencia hay entre <code>_raw.csv</code> y <code>_processed.csv</code> ?	<code>_raw.csv</code> contiene solo las lecturas directas del firmware (ADC y mV). <code>_processed.csv</code> añade <code>gx_est</code> , <code>gy_est</code> , <code>gz_est</code> y <code>g_norm_est</code> . Para análisis mecánico use <i>processed</i> ; <i>raw</i> es útil para verificar saturación del ADC.
¿Para qué sirve el <code>_session.json</code> ?	Contiene todos los parámetros de configuración y los resultados de calidad en formato estructurado. Útil para procesar resultados con Python: <code>data = json.load(open('..._session.json', encoding='utf-8'))</code> .
¿Puedo usar la app con un solo sensor?	Sí. Si solo hay un sensor conectado, el secundario aparecerá con estado <code>fail</code> y razón <code>sensor_missing</code> . Los datos del principal se guardan correctamente. Si el precheck dual falla al esperar ambos, ajuste la duración del precheck a 0 en «Opciones» para saltarlo.